Columnar fields and indexes

WorkTable technical guide — v2, August 2026

Status: Design proposal, circulated for review.

Supersedes: the v1 guide of August 2026. Incorporates `columnar-dsl-review.md` and the grammar in `columnar-dsl-grammar-v2.md`; refines `columnar-index-plan.md`.

> **Additive** — opt in field by field. **Primary key preserved** — every result retains the authoritative key. **Format compatible** — no change to the on-disk format in this slice.

WorkTable's columnar support adds purpose-built column replicas and clustered columnar indexes to the same macro that defines the authoritative row table. Applications keep their primary keys, row-oriented point operations, generated queries, persistence behaviour, and lock model. Selected fields also become available through chunked column scans, ordered lookup, and batch projection.

The result is a practical hybrid: one schema can serve the point-oriented path and the analytical path without introducing a second hand-maintained data model.

> **The primary key remains authoritative and load-bearing.** A `ColumnRowId` is compact supplemental metadata for column position and cross-column alignment. It never replaces the primary key, and it is not a sort position.

Complete example

```
use worktable::prelude::*;
use worktable::worktable;

type DiagnosticBlob = Vec<u8>;   // column types must currently be a single ident

worktable!(
    name: HistoricalCpu,
    persist: true,

    columns: {
        id: u128 primary_key,

        host_id:        u64 columnar,
        captured_at_ns: u64 columnar,
        cpu_percent:    f32 columnar,

        // A wide variable-length column, chunked more finely. 65_536 / 4.
        status: String columnar(chunk_rows(16_384)),

        // Ordinary row-only data remains ordinary.
        diagnostic_blob: DiagnosticBlob,
    },

    columnar_indexes: {
        host_time: {
            cluster_by: [host_id, captured_at_ns],
        },
    },

    // Both entries are defaults and may be omitted; shown for illustration.
    config: {
        columnar_row_id: ColumnRowId32,
        columnar_chunk_rows: 65_536,
    },
);
```

This declaration creates four column replicas. `host_time` provides prefix lookup, range lookup, and clustered traversal by `(host_id, captured_at_ns)`. `status` demonstrates a non-indexed columnar field: it can be scanned and projected even though no columnar index names it. `diagnostic_blob` remains row-only and is reachable only through an explicit row gather.

The three independent choices

1. Which fields are columnar?

Attach `columnar` directly to the field:

```
cpu_percent: f32 columnar,
```

This does not turn the table into a column store. It creates a derived replica for that field while the row store remains the source of truth. The bare form takes table defaults; a field that needs different geometry says so:

```
status: String columnar(chunk_rows(16_384)),
```

An override must be a power-of-two multiple or divisor of `config.columnar_chunk_rows`. That rule exists so chunk boundaries nest: two columns with nested geometry can still be walked as aligned slices, which is what future vector kernels and batched multi-column projection require. Arbitrary per-field sizes would foreclose that quietly, so they are rejected loudly.

`optional` composes. `latency_ms: u64 optional columnar` produces a column with a validity bitmap.

2. Which access paths are clustered?

```
columnar_indexes: {
    host_time: {
        cluster_by: [host_id, captured_at_ns],
    },
},
```

`cluster_by` is the ordered key and the only required member. Every field it names must be columnar. The columns served by the path are derived from `cluster_by` — there is no separate `columns:` list to keep in sync.

Key values are duplicated inside index segments, so the index can prune without touching base columns. Non-key fields are never duplicated; they are gathered from their base column stores by selected identity. Conventional WorkTable indexes coexist with columnar indexes on the same field.

What `cluster_by` does not mean. Base column chunks stay in canonical `ColumnRowId` order. `cluster_by` orders the index, not the base data. The distinction has real performance consequences — see *Access paths and what they cost*.

3. How wide is the row identifier?

A table-level setting in `config`:

```
config: {
    columnar_row_id: ColumnRowId32,
},
```

The identifier is split. Low bits address a slot; high bits carry a generation counter incremented each time that slot is freed.

Type	Live slots	Frees before generation wrap	Intended use
`ColumnRowId16`	4,096	16	Tests and hard-bounded embedded tables only
`ColumnRowId32`	16,777,216	256	**Default.** General-purpose
`ColumnRowId64`	281,474,976,710,656	65,536	Explicit very-large logical range

The split is invisible to callers — `ColumnRowRef` is opaque — so the balance can change without breaking users.

The width is a capacity contract on *simultaneously live* rows. WorkTable does not guess, widen the type, or silently migrate. When the range is exhausted, insertion returns:

```
WorkTableError::ColumnRowIdExhausted(bits)
```

The failed insert is rolled back and existing rows remain valid. WorkTable never wraps, truncates, evicts another row, or panics because the configured range was exceeded.

Because peak concurrent live rows is the hardest quantity in any system to predict, two observability methods ship alongside so applications can alarm before they hit the ceiling rather than discover it:

```
table.columnar_slots_in_use()     -> usize
table.columnar_slots_high_water() -> usize
```

Why the identifier carries a generation

This is the correction that most changes the design from v1, and reviewers should look at it closely.

A bounded identifier must reuse slots — that is what makes it bounded. Reuse creates an ABA hazard. Consider a retained reference across a delete and a reinsert of the *same* primary key, which is routine in every bounded-window workload:

```
t0  insert   pk=938271        -> slot 41207
t1  reader   retains a reference to that row
t2  delete   pk=938271        -> slot 41207 freed
t3  reinsert pk=938271        -> allocator returns slot 41207
t4  reader   uses the retained reference
```

Matching `{ primary_key, slot }` does not help at `t4`: both match exactly. The reader would observe a different row generation as if it were its own. Matching on generation as well makes the stale reference detectable, and `t4` fails cleanly instead of silently succeeding.

Generation tags alone still leave a wrap window, so they are paired with **deferred reclamation**: a freed slot does not return to the allocator while any reader that could hold a reference to it is still in flight. Both mechanisms are required; neither is sufficient alone.

The corollary is that a `ColumnRowId` is stable for one row's lifetime **within one process incarnation**. Columnar state is derived and rebuilt on load, so identifiers are reassigned across a restart. `ColumnRowRef` therefore does not implement `Serialize`, and carries an incarnation epoch so a reference that somehow crosses a restart fails loudly rather than resolving to an unrelated row.

Generated API

Two namespaced accessors, rather than flat per-column and per-index methods. Flat naming would put column names and index names in one namespace, where a table with a column and an index of the same name produces a duplicate-method error pointing at generated code.

```
// Direct scan of any columnar field, indexed or not.
table.columnar()
    .cpu_percent()
    .scan_batches(|batch| aggregate(batch.values(), batch.selection()))?;

// Filter a non-indexed column. This is a full column scan, not an indexed lookup.
table.columnar()
    .captured_at_ns()
    .filter_range(start..end)
    .scan_batches(|batch| consume(batch.values(), batch.selection()))?;

// Clustered index: prefix equality, range, then batch projection.
table.columnar_index()
    .host_time()
    .host_id_eq(42)
    .captured_at_ns_range(start..end)
    .project(|c| (c.cpu_percent(), c.status()))
    .scan_batches(|batch| {
        aggregate(batch.cpu_percent(), batch.status(), batch.selection());
    })?;
```

Three properties are worth naming.

Batches, not vectors. Scans expose typed slices plus a selection bitmap. They do not materialize a `Vec` of the column, and they never materialize `HistoricalCpuRow`. Operating on slices is the difference between a column store and a vector of rows.

Prefix lookup works. Predicate methods are generated per `cluster_by` column — `<column>_eq` and `<column>_range` — so "all rows for this host" is expressible. A composite key without prefix lookup is a single-value key with extra steps. Adding a column to `cluster_by` adds a method rather than changing an arity, so existing callers keep compiling and keep meaning what they meant.

Multi-column projection is one operation. A projection acquires chunk read locks for all referenced columns in deterministic `(column_id, chunk_id)` order. A result set therefore cannot combine `cpu_percent` from before an update with `status` from after it. Two independent single-

column projections would have no such guarantee, which is why projection takes a column tuple rather than being called once per column.

Reaching a non-columnar field requires an explicit gather, named so the cost is visible:

```
let rows = table.columnar_index()
    .host_time()
    .host_id_eq(42)
    .collect_rows()?;          // fetches diagnostic_blob from DataPages
```

Access paths and what they cost

The most important performance fact about this design, stated plainly because v1 left it to inference.

Base column chunks are addressed positionally, in allocation order. A clustered index orders by its key, which is a different order. So an ordered traversal walks a *permutation* of chunk positions — that is random access, and it does not get prefetch or SIMD benefits. What it does get is a much smaller cache footprint than a row scan, because it touches one column instead of every field.

Three paths, three cost profiles:

Path	Access pattern	Benefit	Cost
Direct column scan	Sequential within a column	Prefetch-friendly, vectorizable	No pruning; touches every live value
Clustered index + projection	Prune, then gather by identity	Selectivity; skips irrelevant segments	Gathered access into base chunks
Row gather (`collect_rows`)	Identity → primary key → row link	Reaches non-columnar fields	Full row materialization

Which is fastest depends on selectivity, chunk geometry, and whether the data is warm. The nested `chunk_rows` rule exists so that an aligned fast path stays constructible for the projection case; whether it is worth building is a benchmark question, and the aligned-versus-gathered measurement is a prerequisite to any performance claim in this document.

Ordering guarantee. `cluster_by` orders within a segment. Late inserts and multiple sealed segments may overlap, so a traversal is not globally sorted unless it merges ordered segment streams or performs a final sort. What is guaranteed is that a scan returns no duplicate live rows. v1's unqualified "clustered traversal in key order" promised more than the design delivers.

Insert, update, delete, and vacuum

The generated mutation path maintains derived columnar state alongside the existing indexes, inside the existing per-key mutation gate:

```
authoritative primary key -> authoritative WorkTable row/link
                          -> ColumnRowId directory
                          -> per-field column chunks
                          -> zero or more clustered columnar indexes
```

- **Insert** allocates an identifier and writes the selected column replicas.
- **Update** retains the identifier for the same primary key and refreshes affected values and clustered keys.
- **Delete** removes column values and clustered entries, then marks the slot for deferred reclamation — it does not become immediately allocatable.
- **Reinsert / upsert** reuse the identifier when the row already exists; a fresh insert allocates.
- **Vacuum** may move the physical row link without changing the primary key or the identifier.
- **In-place archived-field changes** mark the affected field's affected chunks dirty. The next access rebuilds only those chunks from authoritative rows.

That last point is a change from v1, which marked the whole replica dirty and made the next reader pay an O(rows) rebuild triggered by an unrelated write. Per-chunk dirtiness bounds the cost, and applications that would rather schedule it than pay it on a reader can:

```
table.columnar_is_dirty() -> bool
table.rebuild_columnar()  -> Result<(), WorkTableError>
```

Column maintenance happens before the mutation gate is released. Publishing column changes after the gate has been released would let two same-key operations land in reverse order.

Persistence and recovery

The first implementation does not change WorkTable's on-disk format. Row data and existing persistent indexes keep their current formats. Columnar state is derived, omitted from `PersistIndex`, and rebuilt from authoritative rows after load.

That boundary buys three properties:

- existing persisted tables remain format-compatible;
- the primary key and row store remain the recovery authority;
- native column checkpoints can be evaluated with benchmarks before committing to a durable format.

It also has a cost that should be stated: rebuild is O(rows) at startup, and because it is lazy the first analytical query after boot pays it. `rebuild_columnar()` lets an application warm eagerly instead.

Compression is **not** accepted as inert configuration. `compression(none)` is the only policy this release compiles; `auto`, `delta`, `rle`, and `dictionary` are reserved and produce a macro expansion error naming what is supported. v1 accepted all of them and retained them as metadata, which meant a user could write `compression(delta)`, benchmark, measure nothing, and reasonably conclude the columnar path was slow. Real codecs belong on sealed immutable chunks, where updates do not repeatedly recompress hot buffers; each policy is enabled as its codec lands.

Practical patterns

Bounded window (high-frequency trading)

```
worktable!(
    name: OrderBookEvents,

    columns: {
        event_id:     u128 primary_key,
        instrument:   u32 columnar,
        timestamp_ns: u64 columnar,
        price_ticks:  i64 columnar,
    },

    columnar_indexes: {
        instrument_time: {
            cluster_by: [instrument, timestamp_ns],
        },
    },

    config: {
        columnar_chunk_rows: 16_384,
        columnar_row_id: ColumnRowId32,
    },
);
```

Keep order and event identifiers as the authoritative primary key; cluster analytical access by instrument and time.

Note that this recommends the 32-bit default, where v1 recommended a 16-bit identifier for exactly this pattern. Two reasons. A hard live-window bound is the workload that churns slots hardest — constant eviction and insertion is what drives generation wrap and what makes a peak-concurrency capacity contract fail during a burst rather than during testing. And after the generation split, `ColumnRowId16` addresses 4,096 live rows, not 65,536, so the bounded-window argument no longer reaches most of these tables. Three extra bytes per live row removes the whole class of problem.

SaaS telemetry

Take the 32-bit default, cluster by tenant and time, keep large diagnostic payloads row-only. Add a non-indexed columnar field for anything commonly scanned or projected after another index has already identified the rows — a status or duration column earns its replica there without needing an index of its own.

Desktop or embedded catalogue

Choose `ColumnRowId16` only when the domain itself supplies a strict upper bound below 4,096 live rows and the footprint saving is measurable on the target. The smaller token reduces directory and index footprint; it changes nothing about the primary key's representation or behaviour anywhere else in WorkTable.

What this design deliberately avoids

- No table-wide `layout: columnar` switch.
- No replacement or weakening of the primary key.
- No implicit 64-bit row identifier when a bounded workload needs less.
- No claim that configured compression is applied — unimplemented policies do not compile.
- No new columnar disk format in the initial compatibility-first slice.
- No silent overflow, no automatic type widening.
- **No immediate slot reuse**, and no reliance on `{ primary_key, slot }` matching for staleness.
- **No claim of globally ordered scans** — ordering is per segment.
- **No claim of identifier stability across a restart** — derived state is rebuilt on load.
- **No snapshot isolation.** Multi-column projection is internally consistent; a sequence of separate queries is not.

Current and planned surface

In the initial implementation

- per-field chunked column replicas;
- a shared configurable identifier width with generation tagging and deferred reclamation;
- prefix, exact, and range lookup on a clustered key;
- ordered traversal with a stated per-segment guarantee;
- direct column scan and filter, indexed or not;
- batched multi-column projection under a single read guard;
- explicit `collect_rows()` gather for non-columnar fields;
- insert / update / delete / reinsert / upsert / vacuum maintenance;

- per-chunk dirty tracking and explicit rebuild;
- slot occupancy and high-water metrics;
- lazy rebuild after persistent load.

Natural follow-up work

- incremental archived in-place updates rather than chunk rebuilds;
- fixed-width vector kernels over aligned chunk geometry;
- sealed-chunk compression, one codec at a time;
- native column checkpoints, manifests, and recovery watermarks;
- background segment compaction;
- generated aggregation and group-by over typed batches;
- a measured cost model spanning row, conventional-index, clustered-columnar, and base-column scan paths;
- optional Arrow export, layered rather than internal.

Open questions for reviewers

1. **Generation tags versus a retention ban.** If the API forbade retaining a `ColumnRowRef` beyond a read guard, deferred reclamation alone would be sufficient and the full identifier width could address slots. That trades ergonomics for address space. Which is worth more?
2. **Is `ColumnRowId16` worth keeping at all?** At 4,096 live rows the footprint saving is small and it costs a monomorphized codegen path plus its share of the test matrix.
3. **`cluster_by` or `order_by`?** The former is familiar but implies base-data reordering that does not happen. The latter is accurate but unfamiliar.
4. **Should the nested `chunk_rows` rule be a hard error or a warning?** Hard error is proposed here. It forecloses a legitimate use case — a column whose natural geometry genuinely differs — in exchange for keeping the aligned path constructible.
5. **Column type grammar.** The column parser currently requires a single ident, so `Vec<u8>` and `[i64; 10]` need `type` aliases. Extending it is independently useful; is it in scope for this slice or a prerequisite tracked separately?
6. **Where does rebuild run?** Reader, writer, or background worker. Per-chunk dirtiness bounds the cost but does not decide who pays it.

WorkTable's columnar direction is additive by design: applications opt fields and access paths into analytical storage while the proven primary-key and row machinery continues to carry correctness. This revision keeps that framing and tightens three things v1 left loose — identifier safety under reuse, consistency across a multi-column read, and the gap between what the compiler accepts and what the engine actually does.